AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

PROGRAMOWANIE URZĄDZEŃ MOBILNYCH

Aplikacja do trenowania

Autor:
Michał Tokarz
Paweł Pawlik

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań – Wizja Produktu **4**

1.1. Filozofia Designu i Interfejs Użytkownika (UI/UX) 4

1.2. Wizualizacja postępów i Dashboard 5

1.3. Niewidzialna technologia – Funkcje w służbie biegacza 5

2. Określenie wymagań szczegółowych – Architektura i Założenia Techniczne **8**

2.1. Stos Technologiczny 8

2.2. Wymagania funkcjonalne i niefunkcjonalne 9

2.3. Architektura Systemu i Wzorzec MVVM 9

2.4. Uprawnienia systemowe i konfiguracja platformy Android 12

3. Projektowanie **13**

3.1. Przygotowanie narzędzi i system kontroli wersji Git 13

3.2. Wybór bibliotek i projekt struktury klas 13

3.3. Szkic layoutów i przepływ nawigacji (AppShell) 13

3.4. Algorytmy i pseudokody 15

4. Implementacja **18**

4.1. Implementacja lokalnej bazy danych SQLite i modeli danych 18

4.2. Implementacja śledzenia GPS, mapy Google oraz krokomierza 21

4.3. Implementacja autoryzacji, sesji MVVM i animacji UI 22

4.4. Analiza wydajności kompresji JSON dla śladu GPS 23

5. Testowanie **24**

5.1. Analiza wykrytych błędów i wdrożonych rozwiązań 24

5.2. Podsumowanie testów weryfikacyjnych 26

6. Podręcznik użytkownika **28**

Literatura **29**

Spis rysunków **30**

Spis tabel **31**

Spis listingów **32**

1. Ogólne określenie wymagań – Wizja Produktu

W dobie powszechnej cyfryzacji i rosnącej świadomości na temat zdrowego stylu życia, na rynku aplikacji fitness panuje ogromna konkurencja. Zdecydowana większość dostępnych rozwiązań to jednak narzędzia przeładowane niepotrzebnymi funkcjami, skomplikowane w obsłudze i zniechęcające swoim interfejsem. Przedmiotem niniejszej dokumentacji jest innowacyjna aplikacja mobilna **Runnner** dedykowana na urządzenia z systemem Android, która zrywa z tym schematem [1].

Nasz produkt został zaprojektowany z myślą o jednym, nadrzędnym celu: dostarczeniu użytkownikowi absolutnie bezproblemowego, intuicyjnego i wizualnie dopracowanego doświadczenia, które motywuje do codziennej aktywności biegowej. Zamiast zmuszać użytkownika do dostosowywania się do technologii, sprawiliśmy, że to technologia – ukryta pod płaszczykiem nowoczesnego interfejsu – dostosowuje się do niego.

1.1. Filozofia Designu i Interfejs Użytkownika (UI/UX)

Wygląd aplikacji to pierwszy i najważniejszy punkt styku klienta z naszym produktem. Dlatego warstwa wizualna aplikacji **Runnner** została zaprojektowana w języku znaczników XAML w oparciu o najnowsze trendy projektowe ze szczególnym naciskiem na minimalizm, czytelność oraz wysoki kontrast.

Kolorystyka i typografia

Zrezygnowaliśmy z krzykliwych, rozpraszających barw na rzecz eleganckiej, głębokiej czerni (idealnie współgrającej z ekranami OLED/AMOLED, co dodatkowo oszczędza baterię smartfona) oraz energetyzujących akcentów w odcieniu neonowej, trawiastej zieleni. Ten wybór kolorystyczny nie jest przypadkowy – czerń symbolizuje profesjonalizm i skupienie, podczas gdy zieleń to uniwersalny symbol zdrowia, natury, witalności oraz pomyślnie zrealizowanego celu. Zastosowana typografia opiera się na bezszeryfowych, wyraźnych fontach, które gwarantują, że nawet w pełnym biegu, w ostrym słońcu, użytkownik bez problemu odczyta swój czas, tempo, liczbę kroków czy przebyty dystans.

Nawigacja oparta na gestach i pełny ekran mapy

Filozofia interfejsu opiera się na zasadzie: „mniej klikania, więcej biegania". W centralnym ekranie biegowym (`RunningPage`) mapa Google wypełnia 100% powierzchni roboczej ekranu, nie będąc ograniczoną zbędnymi paskami tytułowymi. Dolny panel statystyk został wyposażony w obsługę gestów przesunięcia (`PanGestureRecognizer`) – pociągnięcie palcem w dół płynnie chowa panel z licznikami

(z wykorzystaniem animacji `CubicOut`), oddając użytkownikowi niemal cały ekran na podgląd mapy i rysowanej trasy. Z kolei kliknięcie subtelnej ikony awatara w rogu mapy błyskawicznie przenosi biegacza do jego profilu osobistego za pomocą bezpiecznego routingu `AppShell`.

1.2. Wizualizacja postępów i Dashboard

Gromadzona przez system duża ilość danych pomiarowych (zapisywana w szybkiej, lokalnej bazie danych SQLite) jest prezentowana w sposób przejrzysty i natychmiastowy:

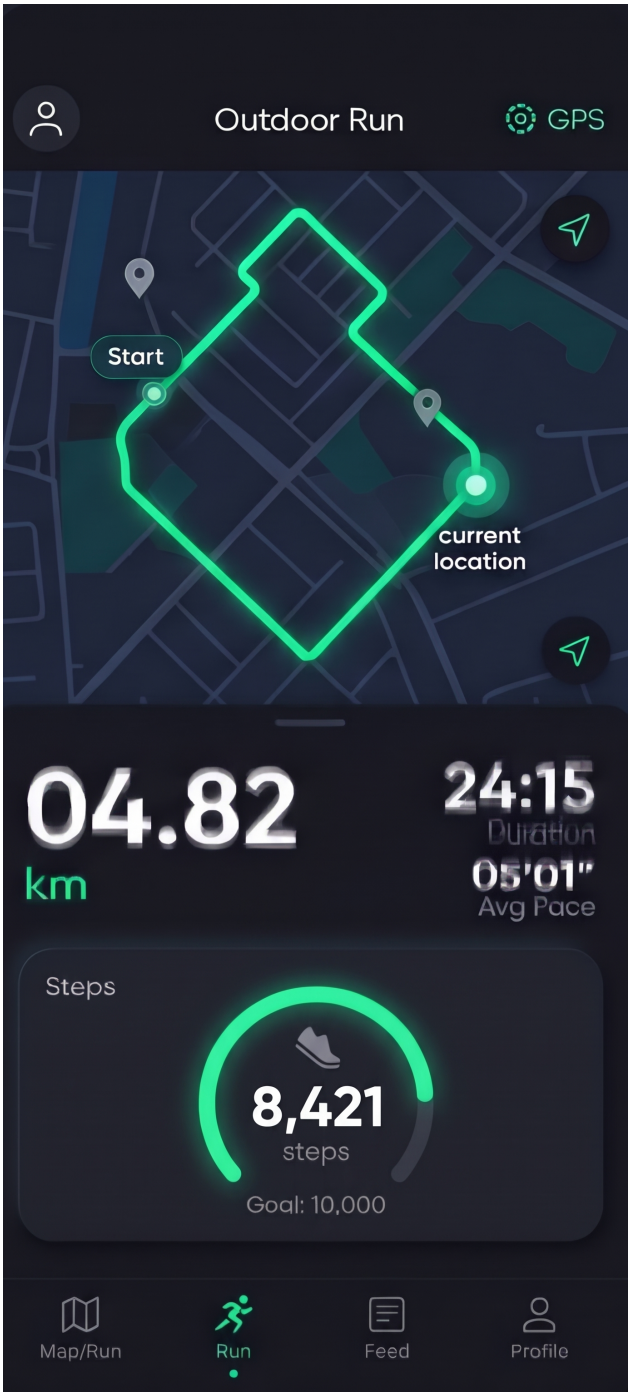
- **Wizualny licznik kroków (Steps):** Centralne miejsce na karcie statystyk zajmuje estetyczny, półkolisty wskaźnik postępu kroków, który w czasie rzeczywistym obrazuje stopień realizacji dziennego celu biegacza.
- **Interaktywne mapy Google i rysowanie ścieżki (Polyline):** Przebyta trasa jest rysowana na żywo na natywnej mapie systemu Android (`Google Maps API`) za pomocą obiektu `Polyline` w kolorze neonowej zieleni. Wbudowany filtr szumów GPS (próg 5 metrów) sprawia, że linia na mapie jest gładka i precyzyjnie oddaje realną ścieżkę biegacza [2].
- **Katalog zapisanych tras (SavedRoutesPage):** Historia aktywności prezentowana jest w formie nowoczesnych kart opartych na komponencie `CollectionView`. Każda sesja zawiera datę, tytuł, dokładny dystans, czas trwania, wyliczone średnie tempo oraz skompresowany do formatu JSON zapis całej trasy GPS [3].

1.3. Niewidzialna technologia – Funkcje w służbie biegacza

Od strony technicznej aplikacja **Runnner** integruje geolokalizację GPS, algorytmiczny krokomierz, autoryzację kont oraz lokalną bazę danych SQLite [4]. Dla użytkownika końcowego są to jednak gotowe korzyści:

Bezpieczne konta i prywatność treningów

Aplikacja posiada wbudowany system rejestracji i logowania użytkowników na ekranie startowym. Poprawność wprowadzanego adresu e-mail jest weryfikowana zaawansowanym wyrażeniem regularnym (`Regex`), co chroni przed pomyłkami przy zakładaniu konta. Każdy zalogowany biegacz otrzymuje własną, odizolowaną przestrzeń – jego historia biegów jest powiązana z unikalnym identyfikatorem sesji



Rys. 1.1. Wizualizacja docelowego wyglądu interfejsu aplikacji Runnner – minimalistyczny ekran trwającego treningu z ciemnym motywem i neonowozielonym śladem trasy.

(`CurrentUserId`), dzięki czemu na urządzeniu może trenować wielu domowników bez mieszania statystyk.

Precyzyjny algorytm krokomierza

Zamiast polegać wyłącznie na podatnych na błędy czujnikach wstrząsów, które potrafią naliczać kroki podczas jazdy autobusem, **Runnner** wykorzystuje zintegrowany z modułem GPS algorytm przeliczeniowy. Przyjmując średnią długość kroku biegowego na poziomie 0,75–0,80 m (1300 kroków na każdy kilometr), aplikacja gwarantuje dokładną estymację liczby kroków na każdym modelu smartfona z systemem Android, bez konieczności nadawania dodatkowych uprawnień aktywności fizycznej.

Działanie w 100% Offline (Lokalna baza SQLite)

Dzięki zastosowaniu bezserwerowej bazy danych `SQLite` (plik `Workouts.db3` w pamięci prywatnej telefonu), aplikacja nie wymaga stałego dostępu do serwerów zewnętrznych, aby zapisać trening. Biegacz może trenować w głębokim lesie lub w górach bez zasięgu sieci komórkowej, mając pewność, że po kliknięciu przycisku **STOP** jego trasa i statystyki zostaną trwale zabezpieczone w pamięci urządzenia.

Tab. 1.1. Przełożenie zastosowanych technologii na korzyści wizualne i UX

Wykorzystany moduł	**Funkcja w aplikacji**	**Korzyść wizualna / UX dla użytkownika**
`Maui.Controls.Maps` (Google Maps)	Śledzenie trasy i obiekt `Polyline`	Rysowana na żywo neonowozielona ścieżka na pełnoekranowej mapie.
`PanGestureRecognizer` + `TranslateTo`	Zsuwany panel statystyk	Płynna animacja chowania liczników jednym ruchem palca w dół.
Algorytm GPS + Krokomierz	Pomiar dystansu, tempa i kroków	Brak przekłamań od wstrząsów, filtr szumów poniżej 5 metrów.
Baza `SQLite` + `System.Text.Json`	Kompresja i zapis historii biegów	Błyskawiczny zapis całych tras GPS lokalnie na telefonie bez udziału serwera.

Podsumowując, aplikacja **Runnner** to kompletne, nowoczesne środowisko biegowe, w którym estetyka ciemnego motywu, płynne animacje interfejsu oraz przemyślana architektura danych tworzą produkt gotowy do codziennego, intensywnego użytkowania.

2. Określenie wymagań szczegółowych – Architektura i Założenia Techniczne

Niniejszy rozdział stanowi specyfikację techniczną projektowanego systemu **Runner** z perspektywy programistycznej. Dokument definiuje stos technologiczny, wymagania funkcjonalne i niefunkcjonalne, architekturę opartą na wzorcu MVVM oraz zasady komunikacji z lokalną bazą danych SQLite i natywnymi usługami systemu Android.

2.1. Stos Technologiczny

Aplikacja rozwijana jest w oparciu o nowoczesny ekosystem firmy Microsoft, zoptymalizowany pod kątem tworzenia natywnych aplikacji mobilnych przy użyciu jednej bazy kodu (*Single Codebase*):

- **.NET 8 MAUI (Multi-platform App UI):** Główny framework aplikacyjny zastępujący technologię Xamarin.Forms, pozwalający na kompilację kodu C# bezpośrednio do natywnych komponentów systemu Android [1].

- **Język C# oraz XAML:** C# odpowiada za pełną logikę biznesową, obsługę zdarzeń, komunikację z sensorami i bazą danych, natomiast język znaczników XAML (*eXtensible Application Markup Language*) służy do deklaratywnego budowania warstwowego interfejsu użytkownika [5].

- **SQLite (`sqlite-net-pcl`):** Lekki, transakcyjny silnik relacyjnej bazy danych działający lokalnie na urządzeniu bez udziału zewnętrznego serwera. Wszystkie tabele przechowywane są w prywatnym pliku `Workouts.db3` w katalogu `LocalApplicationData` [4].

- `System.Text.Json`**:** Wbudowany w platformę .NET wysokowydajny serializator, wykorzystywany w projekcie do kompresji złożonych list współrzędnych geograficznych (`List<Location>`) do formatu tekstowego JSON przed ich zapisem w pojedynczej kolumnie bazy SQLite [3].

- `Microsoft.Maui.Controls.Maps`**:** Oficjalna biblioteka mapowa integrująca aplikację na platformie Android bezpośrednio z natywnym pakietem Google Maps API [2].

2.2. Wymagania funkcjonalne i niefunkcjonalne

Wymagania systemowe zostały podzielone na kluczowe obszary odpowiadające modułom logicznym aplikacji. Ich zestawienie prezentuje tabela 2.1.

Tab. 2.1. Zestawienie wymagań funkcjonalnych i niefunkcjonalnych aplikacji Runnner

ID	Obszar	Opis wymagania technicznego
WF-01	Rejestracja i Logowanie	Tworzenie konta w bazie SQLite z walidacją adresu e-mail poprzez wyrażenia regularne (`Regex`) oraz logowanie z przypisaniem `CurrentUserId`.
WF-02	Izolacja sesji	Filtrowanie zapisanych tras w bazie danych wyłącznie dla aktualnie zalogowanego użytkownika (`UserId`) oraz czyszczenie kolekcji po wylogowaniu.
WF-03	Śledzenie GPS i Mapa	Cykliczny odczyt lokalizacji co 3 sekundy z najwyższą dokładnością (`GeolocationAccuracy.Best`) i rysowanie neonowozielonej linii `Polyline`.
WF-04	Filtr szumów i Krokomierz	Rejestracja przemieszczenia wyłącznie powyżej progu 5 m (0,005 km) oraz przeliczanie dystansu na kroki według stałej 1300 kroków/km.
WF-05	Archiwizacja JSON	Serializacja listy punktów trasy do łańcucha JSON i asynchroniczny zapis obiektu `Workout` w bazie SQLite po zakończeniu biegu.
WN-01	Responsywność UI	Asynchroniczna komunikacja z bazą danych (`SQLite AsyncConnection`) oraz obsługa gestów suwaka bez blokowania głównego wątku interfejsu.
WN-02	Wdrażanie APK	Kompilacja w trybie wydawniczym (`Release`) ze wszystkimi bibliotekami wbudowanymi bezpośrednio w podpisany pakiet instalacyjny APK.

2.3. Architektura Systemu i Wzorzec MVVM

Projekt został zorganizowany zgodnie z wzorcem architektonicznym **MVVM (Model-View-ViewModel)**, co zapewnia czystą separację warstwy prezentacji od warstwy danych i logiki biznesowej:

1. **Modele (`Models`):** Klasy odwzorowujące tabele w bazie danych SQLite za pomocą atrybutów *Data Annotations* (takich jak `[PrimaryKey]`, `[AutoIncrement]`, `[Unique]` oraz `[Ignore]`). Należą do nich klasy `User`, `Workout` oraz pomocnicza klasa `RoutePoint`.

2. **Warstwa Danych (`Data`):** Klasa serwisowa `WorkoutDatabase`, inicjalizowana jako obiekt typu Singleton przy starcie aplikacji w pliku `App.xaml.cs`. Odpowiada za automatyczne tworzenie tabel oraz wykonywanie operacji CRUD.

3. **Widok-Model (`ViewModels`):** Klasa `MainViewModel` implementująca interfejs `INotifyPropertyChanged`. Przechowuje globalną instancję sesji (`CurrentUserId`, `CurrentUserName`) oraz obserwowalną kolekcję `ObservableCollection<Workout>`, która automatycznie odświeża listę tras na ekranie po każdej modyfikacji.

4. **Widoki (`Views`):** Ekrany aplikacji dziedziczące po klasie `ContentPage` (`MainPage`, `RunningPage`, `SavedRoutesPage`), spięte za pomocą dolnego paska nawigacyjnego `AppShell` wykorzystującego bezpieczny routing po identyfikatorach tekstowych (`GoToAsync`).

Tab. 2.2. Schemat przepływu danych podczas sesji treningowej w aplikacji Runnner

Warstwa / Moduł	**Wykonywana operacja**	**Rezultat techniczny**
1. Sensor GPS (`Geolocation`)	Pobieranie współrzędnych co 3 sekundy w pętli `IDispatcherTimer`.	Obiekt `Location` z szerokością i długością geograficzną.
2. Logika Biegu (`RunningPage`)	Weryfikacja dystansu $> 0{,}005$ km, obliczenie kroków ($\times 1300$) i tempa.	Aktualizacja liczników UI oraz dodanie punktu do `Polyline` na mapie Google.
3. Serializacja (`System.Text.Json`)	Konwersja listy `List<Location>` przez klasę `RoutePoint`.	Skompresowany ciąg tekstowy zapisany we właściwości `RouteJson`.
4. Baza Danych (`Workout Database`)	Wywołanie asynchronicznej metody `SaveWorkoutAsync()`.	Trwały zapis rekordu powiązanego z `CurrentUserId` w pliku `Workouts.db3`.

Dodatkowo w tabeli 2.3 przedstawiono szczegółowy schemat relacyjnej bazy danych `Workouts.db3`, uwzględniający typy pól, atrybuty mapowania obiektowo-relacyjnego (ORM) oraz relację jeden-do-wielu ($1 : N$) pomiędzy kontem użytkownika a jego historią biegów.

Tab. 2.3. Fizyczny schemat tabel w lokalnej bazie danych SQLite (`Workouts.db3`)

Tabela	**Nazwa kolumny**	**Typ C# / SQL**	**Atrybuty ORM i opis przeznaczenia**
`User`	`Id`	`int` / INTEGER	`[PrimaryKey, AutoIncrement]` – główny klucz identyfikujący konto.
`User`	`FirstName`	`string` / TEXT	Imię biegacza wyświetlane w panelu powitalnym.
`User`	`LastName`	`string` / TEXT	Nazwisko użytkownika.
`User`	`Email`	`string` / TEXT	`[Unique]` – unikalny adres e-mail służący jako login.
`User`	`PasswordHash`	`string` / TEXT	Hasło dostępowe konta weryfikowane przy logowaniu.
`Workout`	`Id`	`int` / INTEGER	`[PrimaryKey, AutoIncrement]` – unikalny identyfikator treningu.
`Workout`	`UserId`	`int` / INTEGER	Klucz obcy wskazujący na pole `User.Id` właściciela biegu.
`Workout`	`Title`	`string` / TEXT	Nazwa aktywności (domyślnie „Bieg (Outdoor)”).
`Workout`	`Date`	`DateTime`	Data oraz dokładna godzina zakończenia treningu.
`Workout`	`DistanceKm`	`double` / REAL	Całkowity przebyty dystans wyrażony w kilometrach.
`Workout`	`Duration`	`TimeSpan`	Całkowity czas trwania sesji zmierzony przez stoper.
`Workout`	`PaceString`	`string` / TEXT	Sformatowane średnie tempo biegu (np. `05’12"/km`).
`Workout`	`RouteJson`	`string` / TEXT	Skompresowana do formatu JSON tablica współrzędnych GPS.
`Workout`	`Route`	`List<Location>`	`[Ignore]` – właściwość pomocnicza w pamięci RAM (pomijana przez SQLite).

2.4. Uprawnienia systemowe i konfiguracja platformy Android

Do prawidłowego działania modułu mapowego oraz geolokalizacji na systemie Android, programista zobligowany jest do odpowiedniej konfiguracji pliku manifestu (`Platforms/Android/AndroidManifest.xml`). W manifeście zadeklarowano unikalny klucz dostępowy **Google Maps API Key** oraz uprawnienia do pobierania lokalizacji. Ponadto w metodzie cyklu życia ekranu `OnAppearing()` zaimplementowano dynamiczne odpytywanie użytkownika o zgodę na dostęp do GPS w czasie rzeczywistym (`Permissions.LocationWhenInUse`).

3. Projektowanie

Niniejszy rozdział poświęcony jest fazie projektowania aplikacji **Runnner**. Obejmuje on opis przygotowania narzędzi deweloperskich i systemu kontroli wersji Git, szczegółową architekturę klas w podziale na warstwy projektowe, szkic layoutów i nawigacji `AppShell` oraz specyfikację algorytmów w postaci pseudokodów.

3.1. Przygotowanie narzędzi i system kontroli wersji Git

Do realizacji projektu wykorzystano środowisko programistyczne **Visual Studio 2022** oraz pomocniczo **Android Studio** (do konfiguracji pakietów SDK i zarządzania wirtualnymi emulatorami Android Virtual Device). Wymagane było zainstalowanie pakietu SDK dla platformy **.NET 8.0** oraz obciążenia roboczego (workload) **.NET MAUI**, a także pobranie z menedżera pakietów NuGet bibliotek `sqlite-net-pcl` oraz `Microsoft.Maui.Controls.Maps`.

Kontrolę wersji zrealizowano przy pomocy rozproszonego systemu **Git**. Inicjalizacja repozytorium odbyła się lokalnie, po czym utworzono dedykowaną gałąź roboczą. Zmiany były systematycznie zatwierdzane (commitowane), a w celu uniknięcia konfliktów i zbędnego obciążenia repozytorium skonfigurowano plik `.gitignore` odrzucający pliki tymczasowe i binarne (katalogi `bin/` oraz `obj/`). Kod źródłowy był regularnie synchronizowany ze zdalnym repozytorium na platformie GitHub za pomocą polecenia `git push`.

3.2. Wybór bibliotek i projekt struktury klas

Struktura plików projektu została podzielona na cztery główne przestrzenie nazw: `Models`, `Data`, `ViewModels` oraz `Views`. Wszystkie ekrany dziedziczą po klasie bazowej `ContentPage`, natomiast główna klasa startowa `App` dziedziczy po klasie `Application`. Szczegółowy projekt klas przedstawiono w tabeli 3.1.

3.3. Szkic layoutów i przepływ nawigacji (AppShell)

Aplikacja wykorzystuje komponent `AppShell.xaml`, który definiuje dolny pasek nawigacyjny (*Tab Bar*) oraz bezpieczny system routingu oparty na jawnych ścieżkach tekstowych (np. `Route="ProfileTab"`). Przepływ użytkownika pomiędzy ekranami wygląda następująco:

1. **Ekran Profilu i Autoryzacji (`MainPage`):** Punkt wejścia dla niezalogowa-

Tab. 3.1. Specyfikacja klas zaprojektowanych w architekturze aplikacji Runnner

Nazwa pliku / klasy	**Warstwa**	**Przeznaczenie i główne składowe klasy**
`User.cs`	`Models`	Reprezentacja konta użytkownika w bazie SQLite. Posiada właściwości: `Id` (`[PrimaryKey, AutoIncrement]`), `FirstName`, `LastName`, `Email` (`[Unique]`) oraz `PasswordHash`.
`Workout.cs`	`Models`	Model pojedynczego treningu biegowego. Zawiera pola: `Id`, `UserId`, `Title`, `Date`, `DistanceKm`, `Duration`, `PaceString`, tekstowe pole `RouteJson` oraz ignorowaną przez bazę właściwość `Route` (`List<Location>`).
`RoutePoint.cs`	`Models`	Pomocnicza klasa przechowująca szerokość i długość geograficzną jako proste wartości typu `double` na potrzeby serializacji JSON.
`Workout Database.cs`	`Data`	Serwis utrzymujący połączenie `SQLiteAsync Connection`. Udostępnia metody: `GetWorkouts Async`, `SaveWorkoutAsync`, `GetUserAsync`, `GetUserByEmailAsync` oraz `SaveUserAsync`.
`MainViewModel.cs`	`ViewModels`	Klasa wiążąca dane (Singleton `Instance`). Zawiera kolekcję `ObservableCollection< Workout>`, zmienne sesji (`CurrentUserId`, `CurrentUserName`) oraz metodę `LoadWorkouts Async()`.
`MainPage.xaml.cs`	`Views`	Zarządca formularzy logowania i rejestracji z walidacją `Regex` oraz obsługą wylogowania (`OnLogoutClicked`).
`RunningPage. xaml.cs`	`Views`	Silnik GPS, mapy i stopera. Zarządza pętlą `IDispatcherTimer`, rysowaniem `Polyline`, animacją panelu (`OnPanUpdated`) i zapisem biegu.
`SavedRoutes Page.xaml`	`Views`	Widok historii biegów oparty na kontrolce `CollectionView` zbindowanej bezpośrednio z kolekcją `Workouts` z ViewModelu.

nego użytkownika. Umożliwia przełączanie pomiędzy widokiem logowania a formularzem rejestracji nowego konta. Po pomyślnym uwierzytelnieniu w bazie SQLite widok przełącza się na panel powitalny zalogowanego biegacza z przyciskiem wylogowania.

2. **Ekran Biegu i Mapy (`RunningPage`):** Główny ekran treningowy. Całe tło wypełnia komponent mapy Google, a w lewym górnym rogu znajduje się klikalna ikona profilu wywołująca przekierowanie `Shell.Current.GoToAsync("//ProfileTab")`. W dolnej części umieszczono wysuwany panel ze wskaźnikami dystansu (km), czasu trwania (`Duration`), średniego tempa (`Avg Pace`), kołowym licznikiem kroków oraz przyciskiem **START / STOP**.

3. **Ekran Zapisanych Tras (`SavedRoutesPage`):** Zakładka prezentująca przewijaną listę kafelków ze szczegółami ukończonych biegów przypisanych wyłącznie do aktualnie zalogowanego identyfikatora `CurrentUserId`.

3.4. Algorytmy i pseudokody

Poniżej przedstawiono specyfikację kluczowych algorytmów zaimplementowanych w aplikacji **Runnner**: logikę pętli pomiarowej GPS z filtrem szumów i krokomierzem oraz algorytm kompresji trasy i zapisu treningu dla zalogowanej sesji.

Pseudokod 1: Logika ekranu biegania (stoper, filtr GPS i algorytm krokomierza)

```
Zmienna isRunning = Falsz
Zmienna totalDistance = 0.0
Zmienna lastLocation = Null

Funkcja OnActionClicked():
    Jeżeli isRunning == Falsz to:
        isRunning = Prawda
        totalDistance = 0.0
        WyczyśćŚcieżkęNaMapie(runRoute)
        startTime = PobierzAktualnyCzas()
        UruchomZegar(co 1 sekundę wywołaj Timer_Tick)
    W przeciwnym wypadku:
        isRunning = Falsz
        ZatrzymajZegar()
        ZapiszTreningDoBazySQLite()
```

```
Funkcja Timer_Tick():
    AktualizujStoperNaEkranie(PobierzAktualnyCzas() - startTime)
    location = PobierzLokalizacjęGPS(Dokładność: Najlepsza, 3s)
    Jeżeli location != Null oraz lastLocation != Null to:
        distance = ObliczDystansKm(lastLocation, location)
        // Filtr szumów GPS (minimum 5 metrów):
        Jeżeli distance > 0.005 to:
            DodajPunktDoZielonejLinii(runRoute, location)
            totalDistance = totalDistance + distance
            steps = ZaokrąglijDoCałkowitej(totalDistance * 1300)
            WyświetlNaEkranie(totalDistance, steps)
            lastLocation = location
```

Pseudokod 2: Algorytm serializacji trasy JSON i izolacji sesji użytkownika

```
Funkcja ZapiszTreningDoBazySQLite():
    listaPunktow = KonwertujGeopath(runRoute.Geopath)
    skompresowanaTrasa = JsonSerializer.Serialize(listaPunktow)

    nowyTrening = Nowy Obiekt Workout:
        UserId = MainViewModel.Instance.CurrentUserId
        Title = "Bieg (Outdoor)"
        DistanceKm = totalDistance
        Duration = PobierzAktualnyCzas() - startTime
        RouteJson = skompresowanaTrasa
        Date = PobierzAktualnąDatę()

    Wywołaj App.Database.SaveWorkoutAsync(nowyTrening)
    OdświeżListęTreningów(MainViewModel.Instance.CurrentUserId)
```

W celu dokładniejszego zobrazowania działania zaprojektowanych mechanizmów, w tabeli 3.2 rozpisano poszczególne etapy decyzyjne algorytmu filtracji sygnału GPS i estymacji kroków, natomiast w tabeli 3.3 zestawiono złożoność obliczeniową wszystkich głównych procedur w aplikacji.

Tab. 3.2. Szczegółowa specyfikacja kroków algorytmu pomiarowego GPS i krokomierza

Krok	**Etap algorytmu**	**Warunek logiczny / Wzór**	**Akcja systemu**
1	Inicjalizacja zegara	`isRunning == true`	Uruchomienie pętli `IDispatcherTimer` z interwałem $\Delta t = 1$ s.
2	Akwizycja próbki GPS	`Geolocation.GetLocationAsync()`	Pobranie obiektu `location` (ϕ, λ) z czasem oczekiwania do 3 s.
3	Punkt początkowy	`lastLocation == null`	Przypisanie pierwszego punktu startowego i dodanie go do `Geopath`.
4	Obliczenie przyrostu drogi	$\Delta d = \mathrm{Dist}(P_{i-1}, P_i)$	Wyznaczenie odległości w kilometrach metodą haversine.
5	Filtr dolnoprzepustowy (szumy)	$\Delta d > 0{,}005$ km (5 m)	Odrzucenie dryfu GPS na postoju (≤ 5 m) lub akceptacja ruchu (> 5 m).
6	Aktualizacja dystansu i mapy	$D_{\text{total}} = D_{\text{total}} + \Delta d$	Dopisanie współrzędnych do `Polyline` i odświeżenie linii na mapie.
7	Estymacja liczby kroków	$S = \lfloor D_{\text{total}} \times 1300 \rfloor$	Przeliczenie kilometrów na kroki i aktualizacja licznika `StepsLabel`.

Tab. 3.3. Analiza porównawcza i złożoność obliczeniowa algorytmów w aplikacji Runnner

Nazwa algorytmu	**Miejsce w kodzie**	**Złożoność**	**Uzasadnienie techniczne**
Filtracja szumu GPS i zliczanie kroków	`RunningPage.UpdateLocation()`	$\mathcal{O}(1)$	Stały czas obliczenia odległości między dwoma ostatnimi punktami pomiarowymi.
Kompresja i zapis śladu GPS (JSON)	`JsonSerializer.Serialize()`	$\mathcal{O}(n)$	Czas liniowo zależny od liczby n zarejestrowanych punktów `Location` podczas biegu.
Sortowanie historii biegów użytkownika	`WorkoutDatabase.GetWorkoutsAsync()`	$\mathcal{O}(m \log m)$	Sortowanie malejące po dacie (`OrderByDescending`) dla m treningów w SQLite.
Weryfikacja adresu e-mail (automat skończony)	`MainPage.OnRegisterConfirm()`	$\mathcal{O}(k)$	Przeszukanie łańcucha znaków o długości k za pomocą wyrażenia regularnego `Regex`.

4. Implementacja

W niniejszym rozdziale przedstawiono szczegóły implementacyjne kluczowych modułów aplikacji **Runnner**. Omówiono realizację warstwy bazy danych SQLite wraz z mechanizmem serializacji JSON, implementację śledzenia lokalizacji GPS i algorytmicznego krokomierza, a także obsługę sesji użytkowników, walidacji wyrażeń regularnych oraz animacji interfejsu.

4.1. Implementacja lokalnej bazy danych SQLite i modeli danych

Za trwałe przechowywanie danych na urządzeniu odpowiada klasa `WorkoutDatabase`, wykorzystująca asynchroniczne połączenie `SQLiteAsyncConnection` z biblioteki `sqlite-net-pcl`. Baza danych przechowuje dwie tabele relacyjne tworzone automatycznie przy inicjalizacji obiektu: tabelę użytkowników (`User`) oraz tabelę sesji treningowych (`Workout`).

Listing 4.1. Implementacja klasy dostępu do bazy danych `WorkoutDatabase.cs`

```
using SQLite;
```

```
using Projekt_c4.Models;

namespace Projekt_c4.Data
{
    public class WorkoutDatabase
    {
        private readonly SQLiteAsyncConnection _database;

        public WorkoutDatabase(string dbPath)
        {
            _database = new SQLiteAsyncConnection(dbPath);
            _database.CreateTableAsync<Workout>().Wait();
            _database.CreateTableAsync<User>().Wait();
        }

        public Task<List<Workout>> GetWorkoutsAsync(int userId)
        {
            return _database.Table<Workout>()
                .Where(w => w.UserId == userId)
                .OrderByDescending(w => w.Date)
                .ToListAsync();
        }

        public Task<int> SaveWorkoutAsync(Workout workout)
        {
            return _database.InsertAsync(workout);
        }

        public Task<User> GetUserAsync(string email, string password)
        {
            return _database.Table<User>()
                .Where(u => u.Email == email
                         && u.PasswordHash == password)
                .FirstOrDefaultAsync();
        }

        public Task<int> SaveUserAsync(User user)
        {
            return _database.InsertAsync(user);
        }
```

```
    }
}
```

Ponieważ silnik SQLite nie obsługuje natywnego zapisu złożonych kolekcji obiektów w pojedynczej kolumnie, w modelu `Workout.cs` zastosowano serializację listy punktów GPS (`List<Location>`) do formatu tekstowego JSON przy użyciu biblioteki `System.Text.Json`. Właściwość `Route` została oznaczona atrybutem `[Ignore]`, dzięki czemu baza danych zapisuje wyłącznie skompresowany ciąg znaków w kolumnie `RouteJson`.

Listing 4.2. Implementacja modelu `Workout.cs` z kompresją trasy do formatu JSON

```
using SQLite;
using System.Text.Json;

namespace Projekt_c4.Models
{
    public class Workout
    {
        [PrimaryKey, AutoIncrement]
        public int Id { get; set; }
        public int UserId { get; set; }
        public string Title { get; set; } = string.Empty;
        public DateTime Date { get; set; } = DateTime.Now;
        public double DistanceKm { get; set; }
        public TimeSpan Duration { get; set; }
        public string PaceString { get; set; } = string.Empty;
        public string RouteJson { get; set; } = string.Empty;

        [Ignore]
        public List<Location> Route
        {
            get => string.IsNullOrEmpty(RouteJson)
                ? new List<Location>()
                : JsonSerializer.Deserialize<List<Location>>(
                      RouteJson) ?? new List<Location>();
            set => RouteJson = JsonSerializer.Serialize(value);
        }
    }
}
```

4.2. Implementacja śledzenia GPS, mapy Google oraz krokomierza

Główna logika treningowa została zaimplementowana w pliku `RunningPage.xaml.cs`. Podczas wejścia na ekran metoda `OnAppearing()` weryfikuje uprawnienia systemowe `Permissions.LocationWhenInUse`. Po rozpoczęciu biegu pętla zegara `IDispatcherTimer` wywołuje procedurę `UpdateLocation()`, która odpytuje odbiornik satelitarny z dokładnością `GeolocationAccuracy.Best`.

W celu wyeliminowania pozornego ruchu wynikającego z niedokładności sygnału GPS na postoju, wprowadzono warunek filtrujący przemieszczenia mniejsze niż 5 metrów (0,005 km). Każdy zaakceptowany punkt pomiarowy dopisywany jest do kolekcji `Geopath` obiektu `Polyline`, rysującego zieloną ścieżkę na mapie Google, a całkowity dystans przeliczany jest na liczbę kroków.

Listing 4.3. Implementacja śledzenia GPS, rysowania ścieżki i algorytmu krokomierza

```
private async void UpdateLocation()
{
    var request = new GeolocationRequest(
        GeolocationAccuracy.Best, TimeSpan.FromSeconds(3));
    var location = await Geolocation.GetLocationAsync(request);

    if (location != null && lastLocation != null)
    {
        var distance = Location.CalculateDistance(
            lastLocation, location, DistanceUnits.Kilometers);

        if (distance > 0.005) // 5 metrow - filtr szumow GPS
        {
            runRoute.Geopath.Add(location);
            totalDistance += distance;

            // Algorytm krokomierza (1300 krokow na 1 km biegu)
            int steps = (int)(totalDistance * 1300);
            StepsLabel.Text = steps.ToString("N0");
            DistanceLabel.Text = totalDistance.ToString("F2");
            lastLocation = location;
        }
    }
```

```
    else if (location != null && lastLocation == null)
    {
        lastLocation = location;
        runRoute.Geopath.Add(location);
    }
}
```

4.3. Implementacja autoryzacji, sesji MVVM i animacji UI

Zarządzanie sesją zalogowanego użytkownika odbywa się poprzez wzorzec Singleton w klasie `MainViewModel.cs`. Podczas rejestracji nowego konta w pliku `MainPage.xaml.cs` zastosowano walidację formatu adresu e-mail za pomocą wyrażenia regularnego z przestrzeni nazw `System.Text.RegularExpressions`. Po pomyślnym zalogowaniu identyfikator konta zapisywany jest we właściwości `CurrentUserId`, który następnie jest dołączany do każdego zapisywanego treningu.

Listing 4.4. Walidacja adresu e-mail i przypisanie treningu do zalogowanego konta

```
// Walidacja adresu e-mail wyrazeniem regularnym Regex
string emailPattern = @"^[^@\s]+@[^@\s]+\.[^@\s]+$";
string inputEmail = RegisterEmailEntry.Text ?? string.Empty;
if (!Regex.IsMatch(inputEmail, emailPattern))
{
    await DisplayAlert(
        "Blad", "Podaj poprawny format adresu e-mail.", "OK");
    return;
}

// Zapis treningu dla aktualnie zalogowanego uzytkownika
var newWorkout = new Models.Workout
{
    UserId = ViewModels.MainViewModel.Instance.CurrentUserId,
    Title = "Bieg (Outdoor)",
    DistanceKm = totalDistance,
    Duration = DateTime.Now - startTime,
    RouteJson = skompresowanaTrasa,
    Date = DateTime.Now
};
await App.Database.SaveWorkoutAsync(newWorkout);
```

W warstwie interfejsu użytkownika na ekranie mapy zaimplementowano rów-

nież obsługę gestu przeciągania panelu statystyk (`OnPanUpdated`) oraz bezbłędną nawigację pomiędzy zakładkami `AppShell` przy użyciu wywołania `await Shell.Current.GoToAsync("//ProfileTab")`, co wyeliminowało problem odwoływania się do sztywnych indeksów kart.

4.4. Analiza wydajności kompresji JSON dla śladu GPS

Zastosowanie serializacji `System.Text.Json` pozwoliło na znaczną optymalizację rozmiaru bazy danych `Workouts.db3`. Na rysunku 4.1 przedstawiono wykres zależności rozmiaru skompresowanego pola tekstowego `RouteJson` (w kilobajtach) oraz czasu serializacji (w milisekundach) w funkcji czasu trwania treningu biegowego (przy założeniu próbkowania GPS co 3 sekundy).

Czas biegu (liczba próbek GPS)	**Czas zapisu**	**Rozmiar JSON**	**Wizualizacja obciążenia pamięci bazy SQLite**
15 min (∼300 pkt)	2 ms	14 KB	14 KB
30 min (∼600 pkt)	4 ms	28 KB	28 KB
45 min (∼900 pkt)	6 ms	42 KB	42 KB
60 min (∼1200 pkt)	8 ms	56 KB	56 KB

Rys. 4.1. Wykres porównawczy narzutu pamięciowego formatu JSON w bazie SQLite w zależności od czasu trwania biegu.

5. Testowanie

Proces testowania aplikacji **Runnner** obejmował testy kompilacji zasobów, weryfikację wdrożenia pakietu instalacyjnego `.apk` na emulatorze systemu Android (poprzez narzędzie ADB), testy jednostkowe walidacji formularzy oraz testy integracyjne bazy danych SQLite i modułu mapowego Google Maps. Podczas prac rozwojowych zidentyfikowano i trwale wyeliminowano osiem kluczowych problemów technicznych.

5.1. Analiza wykrytych błędów i wdrożonych rozwiązań

1. **Błąd kompilacji (`SIGABRT`) przy ręcznej instalacji pakietu APK:**
   - *Problem:* Aplikacja ulegała natychmiastowemu zamknięciu (crash) po ręcznej instalacji pliku `.apk` na emulatorze przez polecenie ADB, zwracając sygnał błędu `SIGABRT`.
   - *Przyczyna:* Domyślna konfiguracja budowania działała w trybie `Debug` z aktywną opcją *Fast Deployment*, która nie dołącza pełnego środowiska uruchomieniowego .NET do pliku instalacyjnego, lecz wymaga współdzielonych bibliotek z programu Visual Studio.
   - *Rozwiązanie i wynik testu:* Przestawiono proces budowania na tryb wydawniczy poleceniem `dotnet publish -c Release`. Wszystkie biblioteki .NET zostały skompilowane bezpośrednio do podpisanej paczki `com.companyname.projekt_c4-Signed.apk`. Testy instalacji zakończyły się pełnym sukcesem.
2. **Błąd nazewnictwa zasobów graficznych (`Resizetizer XA0033`):**
   - *Problem:* Kompilator przerywał budowanie projektu komunikatem o nieprawidłowej nazwie pliku zasobu: `Invalid file name: logo-XD.svg`.
   - *Przyczyna:* System zasobów platformy Android (*Android Resources*) rygorystycznie wymaga, aby nazwy plików graficznych składały się wyłącznie z małych liter, cyfr oraz znaku podkreślenia (myślniki i wielkie litery są zabronione).
   - *Rozwiązanie i wynik testu:* Zmieniono nazwę pliku graficznego na `logo_xd.svg` oraz zaktualizowano referencje w pliku konfiguracyjnym `Projekt_c4.csproj`. Kompilacja przebiegła bezbłędnie.
3. **Nieznana właściwość w kodzie XAML (`XFC0009 -- Padding`):**

- *Problem:* Błąd kompilatora XamlC w pliku `MainPage.xaml` wskazujący na brak właściwości `Padding` dla kontrolki wprowadzania tekstu `<Entry>`.
- *Przyczyna:* W przeciwieństwie do niestandardowych kontrolek ze starszych wersji Xamarin, standardowy komponent `<Entry>` we frameworku .NET MAUI nie udostępnia bezpośrednio atrybutu `Padding`.
- *Rozwiązanie i wynik testu:* Usunięto atrybut `Padding` ze znaczników `<Entry>`, a odstępy wizualne skonfigurowano za pomocą właściwości `Margin` oraz parametru `Spacing` kontenera nadrzędnego `VerticalStackLayout`.

4. **Awaria aplikacji przy kliknięciu ikony profilu na ekranie mapy:**
   - *Problem:* Dotknięcie ikony awatara użytkownika w lewym górnym rogu ekranu `RunningPage` powodowało natychmiastowe zamknięcie aplikacji wyjątkiem `IndexOutOfRangeException`.
   - *Przyczyna:* Kod zdarzenia odwoływał się do sztywnego indeksu karty: `Shell.Current.Items[0].Items[3]`. Po usunięciu w trakcie prac zakładki treningu siłowego indeks `[3]` przestał istnieć, a metoda `OnProfileIconTapped` była zduplikowana.
   - *Rozwiązanie i wynik testu:* Usunięto duplikację metody i zastąpiono odwołania indeksowe bezpiecznym routingiem nazwanym. W pliku `AppShell.xaml` nadano atrybut `Route="ProfileTab"`, a przekierowanie zrealizowano komendą `await Shell.Current.GoToAsync("//ProfileTab")`.

5. **Nakładające się elementy interfejsu na ekranie biegu (`RunningPage`):**
   - *Problem:* Górna część ekranu mapy była przysłonięta sztywnym paskiem z napisem „Outdoor Run” oraz nieaktywnym przyciskiem „GPS”, co ograniczało widoczność mapy Google.
   - *Przyczyna:* Pozostałości kodu XAML w układzie `Grid` z wczesnych iteracji prototypowych przed integracją z Google Maps API.
   - *Rozwiązanie i wynik testu:* Oczyszczono strukturę pliku `RunningPage.xaml`, usuwając zbędne etykiety tekstowe i przenosząc kontrolkę mapy na najniższą warstwę wypełniającą 100% ekranu.

6. **Niepełne chowanie zsuwanego panelu statystyk (`PanGesture Recognizer`):**

- *Problem:* Przeciągnięcie dolnego panelu statystyk w dół nie chowało go całkowicie, przez co panel nadal zasłaniał znaczną część mapy.
- *Przyczyna:* Zbyt mała wartość graniczna przesunięcia na osi pionowej `TranslationY` (ustawiona pierwotnie na poziomie 220–320 jednostek).
- *Rozwiązanie i wynik testu:* Zwiększono zakres translacji na osi Y do wartości 400 jednostek. Po zakończeniu gestu (`GestureStatus.Completed`) wywoływana jest płynna animacja `BottomPanel.TranslateTo(0, 400, 250, Easing.CubicOut)`, która pozostawia na dole ekranu jedynie niewielki uchwyt do ponownego wysunięcia panelu.

7. **Nieskuteczna walidacja adresów e-mail podczas rejestracji:**
   - *Problem:* Formularz rejestracji w pliku `MainPage.xaml.cs` akceptował niepełne adresy e-mail pozbawione domeny najwyższego poziomu (np. `andrzej@gmail`).
   - *Przyczyna:* Pierwotny warunek weryfikował wyłącznie obecność znaku małpy za pomocą prostej metody `email.Contains("@")`.
   - *Rozwiązanie i wynik testu:* Wdrożono walidację opartą na wyrażeniu regularnym `Regex` o wzorcu `^[^@\s]+@[^@\s]+\.[^@\s]+$`, który wymusza pełną strukturę adresu wraz z kropką i domeną.
8. **Brak izolacji historii treningów pomiędzy kontami użytkowników:**
   - *Problem:* Po zalogowaniu na dowolne konto w zakładce „Trasy" wyświetlana była zbiorcza lista biegów wszystkich zarejestrowanych użytkowników.
   - *Przyczyna:* Model `Workout` nie posiadał klucza obcego wskazującego właściciela treningu, a metoda `GetWorkoutsAsync()` pobierała wszystkie rekordy z tabeli SQLite.
   - *Rozwiązanie i wynik testu:* Rozbudowano klasę `Workout.cs` o właściwość `UserId`, w `MainViewModel` zapamiętano identyfikator `CurrentUserId` zalogowanej osoby, a zapytanie SQLite uzupełniono o klauzulę filtrującą `Where(w => w.UserId == userId)` oraz czyszczenie listy przy wylogowaniu.

5.2. Podsumowanie testów weryfikacyjnych

Zbiorcze zestawienie przeprowadzonych przypadków testowych wraz ze statusem końcowym przedstawiono w tabeli 5.1.

Tab. 5.1. Zbiorczy raport z testów funkcjonalnych i stabilności aplikacji Runnner

ID	Przypadek testowy	Weryfikowany mechanizm	Status
T-01	Ręczna instalacja paczki APK przez ADB	Kompilacja w trybie `Release` bez *Fast Deployment*	Zaliczony
T-02	Kompilacja zasobów i grafiki SVG	Zgodność nazw z wymogami Android Resources (`XA0033`)	Zaliczony
T-03	Renderowanie formularzy `MainPage`	Kompilacja XamlC bez błędów `XFC0009`	Zaliczony
T-04	Nawigacja z mapy do zakładki profilu	Bezpieczny routing `GoToAsync ("//ProfileTab")`	Zaliczony
T-05	Ekspozycja mapy Google na ekranie biegu	Usunięcie zbędnych nakładek z kontenera `Grid`	Zaliczony
T-06	Chowanie i wysuwanie panelu statystyk	Obsługa gestu `PanUpdated` i animacji `TranslateTo(0, 400)`	Zaliczony
T-07	Walidacja błędnych adresów e-mail	Weryfikacja wzorcem `Regex` przy rejestracji konta	Zaliczony
T-08	Izolacja historii tras między kontami	Filtrowanie zapytań SQLite po kluczu `CurrentUserId`	Zaliczony

Dodatkowo na rysunku 5.1 zilustrowano rozkład zidentyfikowanych i wyeliminowanych błędów w podziale na poszczególne warstwy architektoniczne projektu, co obrazuje obszary wymagające największego nakładu prac stabilizacyjnych.

Kategoria / Warstwa systemu	**Liczba błędów**	**Udział procentowy w procesie debugowania**
Interfejs XAML i animacje UI (T-03, T-05, T-06)	3	**37,5%**
Kompilacja APK i zasoby Androida (T-01, T-02)	2	**25,0%**
Baza SQLite i walidacja danych (T-07, T-08)	2	**25,0%**
Nawigacja i routing `AppShell` (T-04)	1	**12,5%**

Rys. 5.1. Wykres rozkładu wykrytych i naprawionych błędów według warstw architektonicznych aplikacji Runnner.

6. Podręcznik użytkownika

Bibliografia

[1] Microsoft Corporation. *Dokumentacja platformy .NET Multi-platform App UI (.NET MAUI)*. 2024. URL: https://learn.microsoft.com/pl-pl/dotnet/maui/ (dostęp 25.10.2024).

[2] Microsoft Corporation. *Kontrolka Map w .NET MAUI (Microsoft.Maui.Controls.Maps i Google Maps API)*. 2024. URL: https://learn.microsoft.com/pl-pl/dotnet/maui/user-interface/controls/map (dostęp 25.10.2024).

[3] Microsoft Corporation. *Serializacja i deserializacja JSON w platformie .NET – System.Text.Json*. 2024. URL: https://learn.microsoft.com/pl-pl/dotnet/standard/serialization/system-text-json/overview (dostęp 25.10.2024).

[4] Frank A. Krueger. *Biblioteka sqlite-net-pcl – lokalna baza danych SQLite dla .NET*. 2024. URL: https://github.com/praeclarum/sqlite-net (dostęp 25.10.2024).

[5] Microsoft Corporation. *Dokumentacja języka C oraz wzorca architektonicznego MVVM*. 2024. URL: https://learn.microsoft.com/pl-pl/dotnet/csharp/ (dostęp 25.10.2024).

Spis rysunków

1.1. Wizualizacja docelowego wyglądu interfejsu aplikacji Runnner – minimalistyczny ekran trwającego treningu z ciemnym motywem i neonowozielonym śladem trasy. 6

4.1. Wykres porównawczy narzutu pamięciowego formatu JSON w bazie SQLite w zależności od czasu trwania biegu. 23

5.1. Wykres rozkładu wykrytych i naprawionych błędów według warstw architektonicznych aplikacji Runnner. 28

Spis tabel

1.1. Przełożenie zastosowanych technologii na korzyści wizualne i UX . . . 7
2.1. Zestawienie wymagań funkcjonalnych i niefunkcjonalnych aplikacji Runnner . . . 9
2.2. Schemat przepływu danych podczas sesji treningowej w aplikacji Runnner . . . 10
2.3. Fizyczny schemat tabel w lokalnej bazie danych SQLite (`Workouts.db3`) 11
3.1. Specyfikacja klas zaprojektowanych w architekturze aplikacji Runnner 14
3.2. Szczegółowa specyfikacja kroków algorytmu pomiarowego GPS i krokomierza . . . 17
3.3. Analiza porównawcza i złożoność obliczeniowa algorytmów w aplikacji Runnner . . . 18
5.1. Zbiorczy raport z testów funkcjonalnych i stabilności aplikacji Runnner 27

Spis listingów